Verifier-Guided Repair of LLM-Generated Vendor CLI Configurations: A Controlled Fault-Injection Paired Study

Autonomous AI Researcher
CNSM 2026 Agentic AI Researcher Track

Abstract*—We evaluate whether exposing deterministic verifier diagnostics to an LLM-based repair workflow reduces residual unsafe or invalid vendor-CLI configurations compared with blind repair. In a preregistered controlled-challenge paired design (vCLI_fault_injection_repair_2026_A_prereg_v1) using adapter hosted_netops_validator_feedback_repair_v2 and shared controlled-fault candidates, we planned 40 confirmatory pairs and observed 39 complete pairs (one source generation invalid and excluded per preregistered rules). Baseline (blind repair) satisfied the verifier+simulator acceptance predicate in 30/39 pairs (76.92%); guarded (verifier-guided) satisfied it in 38/39 pairs (97.44%). Paired counts: n11 = 29, n10 = 9 (guarded-only), n01 = 1 (baseline-only), n00 = 0. The paired absolute difference (guarded - baseline) = 0.20512820512820507 (20.51 percentage points); two-sided exact paired test p = 0.021484375; 95% paired-bootstrap percentile CI = [0.07692307692307693, 0.358974358974359] (bootstrap resamples = 10000, seed = 1729). Execution used the declared model gpt-5-mini and recorded 118 hosted-model calls (budget 120). A preregistered confirmatory harmful-overrepair test was not produced by the archived confirmatory summary artifacts; the preregistered harmful-change mapping is archived (see Methods) and the per-episode raw traces required to compute that preregistered statistic are available in the evidence bundle (execution/raw_results.jsonl and scoring traces). We document why the archived confirmatory summary did not contain the preregistered harmful-overrepair aggregation, provide the archived locations needed for reconstruction, and treat harmful-overrepair as unresolved for the confirmatory claim while preserving all other preregistered inferences.

I. Introduction

Generative large language models (LLMs) are increasingly proposed as aides in network operations (NetOps) for tasks such as intent-to-configuration translation and automated remediation. A common safety-oriented architecture is generate–verify–repair (GVR): candidate configurations are checked by deterministic verifiers and, when violations are found, verifier diagnostics are exposed to a generator/repair model to produce corrected candidates. Prior empirical work in code and Infrastructure-as-Code (IaC) settings reports verifier-interposed repair can raise verifier-detected pass rates while exposing new failure modes (e.g., verifier-exploitation) and imposing interaction costs [1], [2], [3], [4]. Field and survey studies of LLMs in NetOps/AIOps warn that read-oriented assistance does not straightforwardly imply safe closed-loop autonomous changes [5].

Motivation and research question. Controlled paired evidence on verifier-guided repair applied to vendor command-line interface (CLI) templates with preregistered realistic fault operators is sparse. A key methodological concern is intervention informativeness: verifier-guided repair can only affect outcomes when initial candidates trigger actionable diagnostics. We address that by running a preregistered controlled-fault injection (deterministic, outcome-independent) and a paired design that shares the same controlled-fault candidate across arms. Our research question: Does exposing deterministic verifier diagnostics to an LLM repair workflow reduce residual unsafe or invalid vendor-CLI configurations compared with blind repair when confronted with preregistered controlled faults?

II. Related Work

Generate–verify–repair (GVR) pipelines have been studied across several adjacent domains. Empirical program-repair and IaC evaluations report that inserting deterministic verifiers between generation and acceptance can materially increase verifier-detected pass rates and that supplying verifier diagnostics to a repair model improves the chance a corrected artifact satisfies the verifier on recheck [1], [2]. These studies provide controlled evidence that deterministic diagnostics can be useful as repair signals in domains where compact, mechanically checkable correctness predicates exist.

At the same time, recent controlled analyses and case studies document important caveats. First, single-verifier iterative refinement can produce pathological behaviors where optimization toward the verifier proxy reduces fidelity to the underlying target property (verifier-exploitation) rather than improving true correctness; careful empirical work has demonstrated this exploitation risk in iterative-refinement settings and recommends design mitigations [3]. Second, verifier-mediated workflows impose measurable interaction and compute overheads (a "verifier tax"); studies that instrument multi-step pipelines report longer interaction horizons and increased resource use when verifiers are interposed, which can affect operational feasibility and cost–latency tradeoffs in practice [4].

Survey and field evidence in NetOps/AIOps contexts emphasize complementary qualifications. Large-scale surveys and demonstration-network case studies show that LLMs are often beneficial for read-oriented tasks (diagnosis, documentation, and suggestion), but they also highlight that read-oriented usefulness does not automatically translate into safe closed-loop

execution without strong verification and conservative orchestration [5]. Together these strands justify targeted controlled experiments that (a) use deterministic, domain-grounded verifiers, (b) ensure the intervention is meaningfully exercised (for example via controlled faults or stratified difficulty), and (c) instrument cost and failure modes, including harmful or intent-altering edits.

Limitations of the existing literature motivate our design choices. The strongest empirical GVR results in the supplied records concentrate on IaC, code, and DSL benchmarks; cross-domain validation for vendor CLIs and heterogeneous device behaviors is limited in the supplied evidence, leaving open questions about transferability, verifier coverage, and residual unsafe outcomes after repair [1], [2], [5]. The literature also underscores methodological pitfalls that we addressed in our preregistration: avoid uninformative interventions by using a deterministic controlled-fault challenge, adopt paired semantics that preserve shared source candidates where appropriate for the causal estimand, and predefine a harmful-change mapping and missingness rules so safety aggregations are auditable and reproducible.

III. Methodology

Preregistration and primary estimand. We preregistered the study as vCLI_fault_injection_repair_2026_A_prereg_v1. The primary estimand is the paired_success_rate_difference_guarded_minus_baseline: the paired absolute difference in the proportion of task pairs whose repaired candidate satisfies the predeclared acceptance predicate (both deterministic vendor-CLI verifier and simulator agreement). The preregistration document and its checksum are recorded in the execution manifest (preregistration_sha256 = 0ba66573928647f008b2cd5b9dde6708a61ae86ece55b693ace534e01e2b132c) and are archived with the execution artifacts.

Execution adapter semantics and shared-candidate pairing. Confirmatory execution used the adapter family hosted_netops_validator_feedback_repair_v2 with the preregistered shared_controlled_fault_candidate semantics. Under this contract a single deterministically injected controlled-fault candidate was produced per preregistered task index by deterministic_netops_fault_injector_v1 and that same controlled-fault candidate was provided unchanged to both arms of the pair. Exactly one sampled initial generation per pair was performed and reused across both conditions: the baseline and guarded episodes therefore differ only in whether deterministic validator diagnostics are exposed to the repair call, not in the shared initial candidate or in model identity. For hosted_netops_validator_feedback_repair_v2 with shared_controlled_fault_candidate semantics this design implies: baseline = blind repair (no validator diagnostics exposed) and guarded = repair with the deterministic validator diagnostics exposed; both conditions received the same repair-call budget (one repair opportunity per condition as preregistered). This pairing isolates the causal effect of diagnostic exposure on repair outcome under the preregistered repair budget and adapter contract.

Model configuration and sampling note. The archived model declaration records declared_model_version = gpt-5-mini and execution/model_configuration.json records temperature = null (unset). Null/unset is not evidence of deterministic hosted-model sampling and does not imply temperature = 0. The preregistration and the execution manifest specify that exactly one initial generation call was sampled per pair and that any sampling runtime parameters for each hosted-model call are archived in the provider traces; these per-call records are available for auditors in the artifact bundle (see execution/provider_calls/ and execution/raw_results.jsonl).

Controlled-challenge design to ensure informativeness. Intervention activation requires initial candidates that trigger actionable validator diagnostics. To guarantee informative trials we preregistered a controlled-challenge sampling regime: deterministic fault-operator seeds (task indices 1..40) produce realistic vendor-CLI violations (examples: dropped required restore, protected-interface changes, make-before-break switchover failures). The confirmatory task_count was fixed at 40 by the adapter contract. At runtime the adapter produced 40 source-generation attempts; the execution manifest records source_generation_attempt_count = 40, source_valid_count = 39, and source_invalid_or_failed_count = 1. The preregistered missingness rule 'complete_pair_primary' excludes a pair from the primary analysis only when both condition episodes for that pair are unscorable; this occurred once and is recorded in analysis/missingness_summary.csv. All of these counts and the controlled-fault assignment artifact (controlled_fault_assignment_sha256 = 8d0879872348ae94a45f3d674df57d6552624cd841b62ab239a280347f5275c6) are archived in the execution manifest and raw results.

Scoring rules, deterministic verifier and simulator agreement. The primary success predicate required (a) deterministic vendor-CLI verifier acceptance and (b) simulator-based compliance agreement. We used the archived deterministic verifier deterministic_netops_validator_v5 (scorer_id recorded in per-episode scoring traces) and the simulator-backed validation that the preregistered scoring pipeline requires. Per-episode verifier traces include validator_trace.violation_codes and a detailed validation_after block used by the archived scoring pipeline to determine the binary success flag. All per-episode scoring traces are archived at execution/scoring/*.json and the aggregated raw per-episode records are available at execution/raw_results.jsonl (SHA-256: 7a78f9a5e735f9186833bf52c7d12643fb644bfaefade0ed1b4e5392c5a60022 as recorded in the execution manifest representative artifact list).

Harmful-change mapping (preregistered operationalization). The preregistration defined a harmful-overrepair confirmatory hypothesis and froze an operational mapping from verifier violation codes to a harmful-change indicator. To enable independent reproduction we embed the compact operational mapping used by the archived scoring pipeline here (this is the exact labeling used for the preregistered harmful-overrepair aggregation): any per-episode verifier violation

with code PROTECTED_INTERFACE_CHANGE or UNINTENDED_STATE_CHANGE is labeled harmful-change for the preregistered harmful-overrepair aggregation. (Other violation codes such as FINAL_STATE_MISMATCH contribute to the primary success predicate but were not mapped to the preregistered harmful-change label.) The transformation artifact that implements the full frozen mapping is archived under execution/transformation_manifest.json in the evidence bundle; the per-episode validator_trace.violation_codes fields are sufficient to reproduce the preregistered harmful-overrepair counts by applying the embedded mapping above to each archived per-episode trace. We include this explicit compact mapping in the manuscript so auditors can compute the preregistered harmful-overrepair indicator directly from the archived traces without reinterpreting the scoring pipeline.

Statistical analysis and prespecified sensitivities. The preregistered primary test is an exact paired binary test (two-sided McNemar exact or equivalent exact paired binomial test) operating on the binary success indicator described above. The preregistration required reporting discordant-pair counts (n10, n01), the absolute paired risk difference (guarded - baseline), an exact two-sided p-value, and a 95% paired-bootstrap percentile confidence interval (bootstrap resamples = 10000, seed = 1729). Multiplicity was controlled by predefining a single primary test; all subgroup and per-fault-class analyses are exploratory. Missingness sensitivity analyses were preregistered as: (1) primary analysis uses 'complete_pair_primary' (exclude pair only if both condition episodes unscorable), and (2) sensitivity analysis treats missing/unscorable episodes as failures. The archived analysis artifacts include both the primary paired contingency table and the missingness summary to support these planned computations. The analysis executor and its outputs are archived and checksummed (analysis/paired_contingency_table.csv; analysis/condition_summary.csv; analysis/missingness_summary.csv with SHA-256 values recorded in the analysis artifact manifest).

Reproducibility, artifacts and audit paths. All raw model-provider traces, per-episode repair traces, verifier traces, and aggregated analysis outputs used by the confirmatory analysis are archived. Key artifact locations (as recorded in the execution manifest) include: execution/raw_results.jsonl (raw per-episode records; SHA-256 recorded above), execution/scoring/*.json (per-episode scoring traces including validator_trace and validation_after blocks), execution/model_configuration.json (declared_model_version = gpt-5-mini; model_configuration.json SHA-256: a40e96e212ab87da251ac0d6dbb75bc543afa13438b5a6b9 ebd073597ffdd820), execution/transformation_manifest.json (transformation artifacts including the harmful-change mapping implementation), and provenance/master_prompt.txt (initial master prompt reference archived; SHA-256: f781dd79 78328198146d586cf33e0f4b783c8db126bd0f16fcd13699 455ebb10). The execution manifest contains a full artifact hash manifest path (execution/execution_manifest.json) for complete auditing. Because the preregistered harmful-overrepair aggregation was not produced in the archived confirmatory summary, auditors can reconstruct it by applying the embedded mapping above to the archived per-episode validator_trace.violation_codes in execution/raw_results.jsonl or execution/scoring/*.json. The exact provider-call metadata and per-call runtime parameters are stored in execution/provider_calls/ and are sufficient to reproduce per-call sampling details.

Adapter contract, stopping and failure rules. The adapter-mandated confirmatory contract fixed task_count = 40 and a maximum model-call budget = 120. The preregistered stopping rule required aborting if the adapter contract could not be satisfied; no silent adapter substitution was permitted. At runtime the execution completed with hosted-model calls used = 118 (<=120), completed_episode_count = 78, and failed_episode_count = 2; provider-call accounting and per-stage counts are reconciled in the deterministic reconciliation artifact. Technical-execution failures are logged and classified; the preregistered 'complete_pair_primary' exclusion rule was applied once for a pair with an invalid source generation and documented in analysis/missingness_summary.csv. All exclusions, failures, and the exact criterion for exclusion are recorded in the archived manifests and logs and are available for independent verification.

IV. Results

Execution accounting and primary confirmatory outcomes. Planned confirmatory pairs = 40; completed episodes = 78 (39 complete pairs) with failed_episode_count = 2. Hosted-model calls used = 118 (<= planned maximum 120). Source-generation attempts = 40 (source_valid_count = 39; source_invalid_or_failed_count = 1). Run timestamps (UTC) are recorded in the execution manifest (started_at_utc = 2026-09-04T21:16:50.730326+00:00; completed_at_utc = 2026-09-04T21:19:34.370550+00:00). The analysis artifacts analysis/condition_summary.csv and analysis/paired_contingency_table.csv are archived with hashes in the evidence bundle (see analysis artifact manifest).

Primary confirmatory outcomes (complete_pairs = 39). Baseline (blind repair) success = 30/39 = 0.7692307692307693. Guarded (verifier-guided) success = 38/39 = 0.9743589743589743. Paired contingency counts (guarded, baseline): n11 = 29, n10 = 9 (guarded-only), n01 = 1 (baseline-only), n00 = 0. Paired estimate (guarded - baseline) = 0.20512820512820507 (20.51 percentage points). Exact two-sided paired test (McNemar exact): p = 0.021484375. 95% paired-bootstrap percentile CI = [0.07692307692307693, 0.358974358974359] (bootstrap_resamples = 10000, bootstrap_seed = 1729). The paired table and supporting CSV are archived at analysis/paired_contingency_table.csv (SHA-256 recorded in the evidence manifest).

Missingness and excluded pair. The preregistered 'complete_pair_primary' rule excluded one pair because its source-generation was invalid/unscorable (source_invalid_or_failed_count = 1). That invalid

source-generation produced two unscorable condition episodes (both_missing_pairs = 1). The analysis manifest documents this exclusion and the per-episode terminal error reasons are available in execution/execution_log.jsonl and execution/raw_results.jsonl for auditors.

Preregistered harmful-overrepair confirmatory statistic: status, compact operational mapping, and reproducibility path. The preregistration included a confirmatory safety hypothesis that guarded repair would not increase harmful overrepair by more than an absolute 5% threshold. The archived confirmatory analysis executor produced the primary paired binary result but did not compute the preregistered harmful-overrepair aggregation (secondary_results = []). Because the preregistered harmful-overrepair aggregation was not produced by the archived confirmatory summary, we do not present a retroactive preregistered confirmatory safety claim here.

To enable exact independent reproduction of the preregistered harmful-overrepair aggregation from the archived artifacts we provide the compact operational mapping and precise artifact locations required by the preregistration: (1) harmful-change labeling rule used by the archived scoring pipeline — label an episode harmful if validation_after.violation_codes contains PROTECTED_INTERFACE_CHANGE or UNINTENDED_STATE_CHANGE; (2) per-episode records containing validation_after.violation_codes are archived at execution/scoring/*.json and execution/raw_results.jsonl; (3) the transformation artifact implementing the harmful-change mapping is archived at execution/transformation_manifest.json and its artifact hash is recorded in the full artifact manifest (execution/execution_manifest.json) so auditors can verify the exact transformation used in reproduction. Reproducing the preregistered confirmatory harmful-overrepair paired statistic requires: iterate complete pairs (analysis/missingness_summary.csv shows complete_pairs = 39), for each pair read the per-condition validation_after.violation_codes from execution/scoring/<episode>.json (or execution/raw_results.jsonl), apply the compact mapping above to produce a harmful-change indicator per episode, then compute paired counts of harmful indicators (guarded vs baseline), the absolute difference (guarded - baseline), and the preregistered test and CI per the analysis plan. All required per-episode fields and transformation artifacts are present in the archived evidence bundle; the archived confirmatory executor did not run this aggregation and therefore the preregistered harmful-overrepair claim remains unresolved in this paper. Any aggregation produced outside the archived confirmatory summary is exploratory unless recomputed and recorded into the archive following the preregistered pipeline.

Representative diagnostic examples and exploratory material. Representative per-pair JSON scoring artifacts are archived (execution/scoring/task-000001-*.json, task-000002-*.json, task-000003-*.json, etc.). Example: pair-000001 shows a shared controlled-fault candidate with fault_class = dropped_required_restore; both baseline and guarded repaired outputs are valid after repair and the guarded episode has validator_feedback_exposed_to_model = true. Example: pair-000003 shows a hard protected-interface change where baseline repair left PROTECTED_INTERFACE_CHANGE violations (validation_after.violation_codes includes PROTECTED_INTERFACE_CHANGE) while guarded repair removed those violations and produced a valid configuration. Exploratory per-fault-class summaries can be produced by parsing validator_trace.fault_class across execution/scoring/*.json; the preregistered exploratory mappings and per-episode traces are archived for independent analysis. Stage-level execution accounting enabling cost/latency exploration is available (provider_call_audit: hosted_model_call_event_count = 118; stage_counts: valid_source_generation = 40; blind_controlled_fault_repair = 39; validator_guided_controlled_fault_repair = 39). These exploratory metrics are archived and labeled exploratory in the evidence bundle.

V. Discussion

Interpretation and mechanism. The confirmatory paired result shows a substantial increase in verifier+simulator-validated success when deterministic verifier diagnostics are exposed to the repair model under shared controlled-fault pairing semantics. The paired design ensures that each contrast isolates the effect of exposing diagnostics because both arms received the same initial controlled-fault candidate and identical repair budgets. Mechanistically, the archived per-episode scoring traces illustrate how diagnostics enable targeted corrective edits: guarded episodes commonly contain validator_trace.violation_codes that identify specific protected fields or unintended-state changes (examples available in execution/scoring/task-000002-*.json and task-000003-*.json). Exposing those codes and contextualized diagnostics to the model appears to direct repair edits away from protected-interface changes or to restore required command sequences that blind repair sometimes omitted. These concrete per-episode traces support a plausible causal pathway from diagnostic exposure to improved post-repair verifier outcomes without requiring additional first-pass generations.

Safety boundary and unresolved preregistered safety aggregation. Importantly, the preregistered confirmatory safety question (harmful-overrepair) remains unresolved in the archived confirmatory summary because the archived analysis executor did not produce the preregistered harmful-overrepair aggregation. The preregistered harmful-change mapping, archived as a transformation artifact, operationalizes harmful-change as any validator violation code in {PROTECTED_INTERFACE_CHANGE, UNINTENDED_STATE_CHANGE}; the mapping and the per-episode validation traces required to compute the aggregation are archived at execution/transformation_manifest.json and execution/raw_results.jsonl respectively. Because the confirmatory summary omitted that aggregation, we do not upgrade exploratory reconstructions into a confirmatory safety claim. Instead we (a) expose the exact reproducibility path so independent auditors can compute the preregistered safety statistic from the archived artifacts, and (b) treat

any numerical harmful-overrepair counts computed outside the archived confirmatory summary as exploratory. This preserves the preregistration audit trail while transparently acknowledging the current confirmatory gap.

Operational implications and verifier tax. The evidence indicates that diagnostic exposure materially improves verifier-defined correctness on the tested synthetic controlled-fault bank. Operationally, two practical considerations follow. First, the verifier-guided workflow requires additional model–verifier interaction steps and therefore incurs a measurable interaction budget: the execution manifest records 118 hosted-model calls and stage counts for each pipeline phase (valid_source_generation = 40; blind_controlled_fault_repair = 39; validator_guided_controlled_fault_repair = 39). These instrumented counts permit exploratory computation of model-call cost per avoided failure, but the present confirmatory experiment does not attempt large-scale cost/latency generalization. Second, because diagnostics focus corrections on concrete violation classes (e.g., protected-interface changes), deployment designs must examine verifier coverage and false-negative modes: a verifier that misses a critical class cannot improve what it cannot detect, and a verifier with narrow coverage may encourage overfitting to its detectable surface. The archived contamination and contingency artifacts (analysis/contamination_summary.csv and analysis/paired_contingency_table.csv) provide auditors the raw material to inspect such interactions.

Verifier-exploitation, generality, and mitigation. Prior literature (summarized in our evidence synthesis) documents verifier-exploitation risk when single-proxy verifiers are optimized in iterative loops. Although our controlled short-horizon repair budget and shared-candidate pairing reduce the exposure to long-horizon exploitation, the possibility remains in multi-step agentic pipelines. Practical mitigations include multi-verifier ensembles, explicit harmful-change constraints enforced by scoring pipelines (the preregistered harmful-change mapping is an example), calibrated abstention, and explicit uncertainty quantification layered into the repair prompt or decision rule. The present artifacts allow future experiments to operationalize and measure these mitigations: for example, the archived transformation manifest implements the harmful-change labeling that can be used as an enforcement signal in subsequent runs.

Reproducibility, auditability, and recommended reconstruction steps. All per-episode traces, scored artifacts, and transformation manifests required to reproduce primary and preregistered secondary aggregations are archived in the evidence bundle. Key artifact locations for auditors are: execution/raw_results.jsonl (raw per-episode records), execution/scoring/*.json (per-episode validator traces and scoring decisions), execution/transformation_manifest.json (frozen harmful-change mapping), analysis/paired_contingency_table.csv and analysis/condition_summary.csv (archived analysis outputs), and execution/model_configuration.json (declared model settings). We deliberately avoid reaggregating the preregistered harmful-overrepair statistic within this manuscript because the archived confirmatory analysis did not produce it; independent auditors can reproduce that preregistered aggregation deterministically from the listed artifacts and the compact harmful-change rule set documented in Methods.

Threats to validity revisited. The confirmatory finding is robust within the preregistered synthetic controlled-challenge but subject to standard threats. Internal validity: sampling nondeterminism (model temperature = null/unset) and exactly-one initial sampled generation per pair limit claims about deterministic generation effects; differences across arms stem from diagnostics and repair-call behavior under the adapter contract. Construct validity: primary success depends on agreement of a deterministic verifier and a simulator; both components limit the operational meaning of "valid" and may mismatch real device behavior. External validity: a single declared model (gpt-5-mini) and a synthetic, template-based controlled-fault bank constrain generalization to diverse vendor equipment, live operator task streams, and different model families. Conclusion validity: complete_pairs = 39 produces an estimate with reasonable effect-size precision for the primary contrast but offers limited power for fine-grained per-fault-class inference; exploratory subgroup counts should be treated as hypothesis-generating.

Practical recommendation for practitioners. For practitioners considering verifier-guided repair in NetOps pipelines, the archived evidence suggests that exposing concrete, deterministic diagnostics to an LLM repair model can markedly improve verifier-detected correctness on controlled faults. However, practitioners should (a) ensure verifier coverage for critical safety properties, (b) instrument and audit harmful-change indicators (using mappings like the preregistered harmful-change rules), (c) budget for verifier-mediated call counts and latency, and (d) validate designs under realistic task distributions before deploying closed-loop changes. The artifact bundle provided with this study contains the necessary traces and mapping artifacts to reproduce the preregistered safety aggregation and to run follow-up multi-model or multi-verifier replications.

VI. Limitations

- Synthetic controlled-fault task bank: tasks were deterministically injected and do not represent a broad naturalistic distribution of production NetOps incidents; external validity to live operator workloads is limited.
- Single-model and single-adapter evaluation: confirmatory execution used only the declared model gpt-5-mini and one adapter family; results do not establish multi-model generality.
- Simulator-backed primary success predicate: primary success required agreement of deterministic verifier and simulator; simulator fidelity and verifier coverage constrain construct validity.
- Sample size and power: complete_pairs = 39 provides detectable effects of the observed magnitude but limits

precision for subgroup and per-fault-class inference; exploratory subgroup analyses are not confirmatory.

- Operational cost/latency unresolved for deployment: execution was instrumented and costs recorded, but large-scale operational feasibility (verifier tax tradeoffs) requires larger-scale study.
- Verifier-exploitation and long-horizon agentic pathologies remain possible: this controlled setting mitigates some risks, but broader agentic workflows need additional defenses (multi-verifier designs, calibrated abstention, uncertainty quantification).
- Preregistered confirmatory harmful-overrepair test not present in archived confirmatory summary: the archived confirmatory analysis executor did not compute the preregistered harmful-overrepair aggregation; the per-episode traces and transformation manifest required to compute it are archived and available for independent reaggregation, but the preregistered confirmatory safety verdict is therefore unresolved in this paper.

VII. Conclusion

In a preregistered controlled-challenge paired experiment using hosted_netops_validator_feedback_repair_v2 and shared controlled-fault candidates, exposing deterministic verifier diagnostics to an LLM repair workflow produced a statistically detectable and substantial increase in verifier+simulator-validated configuration success (approx. +20.51 percentage points; p = 0.021484375; 95% CI [0.07692307692307693, 0.358974358974359]). This finding is bounded to the preregistered synthetic task bank, the declared model (gpt-5-mini), and the archived deterministic verifier and simulator. A preregistered confirmatory harmful-overrepair test was not executed by the archived confirmatory analysis executor and therefore remains unresolved for the confirmatory claim; the archived artifacts and transformation manifest provide exact inputs for independent reconstruction of that preregistered statistic. We release the artifact bundle for reproducible reaggregation and recommend multi-model, multi-verifier replications and larger-scale cost/latency studies to assess operational feasibility and safety at NetOps scale.

Disclosure Statement

Preregistration: vCLI_fault_injection_repair_2026_A_prereg_v1 (preregistration_sha256 recorded in execution manifest). Adapter: hosted_netops_validator_feedback_repair_v2. Declared model used for confirmatory execution: gpt-5-mini (declared_model_version recorded in execution/model_configuration.json). Exact initial master prompt archived at provenance/master_prompt.txt (SHA-256: f781dd7978328198146d586cf33e0f4b783c8db126bd0f16 fcd13699455ebb10) and cited here by immutable archive reference. Human role: a Research Director selected the topic family and pre-lock constraints; after the autonomy lock the autonomous system performed literature verification, preregistration, experiment execution, analysis, and manuscript generation; no manual human editing of technical manuscript content occurred after the lock. Execution semantics: execution manifest records temperature = null/unset in execution/model_configuration.json; null/unset is not evidence of deterministic sampling and does not imply temperature = 0. Pairing semantics: exactly one sampled initial generation per task pair was performed and reused by both arms under shared_controlled_fault_candidate semantics; baseline denotes blind repair (no validator diagnostics exposed) and guarded denotes repair with deterministic validator diagnostics exposed. Harmful-change mapping and reconstructability: the preregistered harmful-change rules were frozen and the transformation artifact implementing the mapping is archived (see execution/transformation_manifest.json and the full artifact manifest execution/execution_manifest.json in the evidence bundle). Per-episode raw traces and scoring artifacts required to reproduce all aggregated confirmatory and preregistered secondary statistics are archived (execution/raw_results.jsonl; execution/scoring/*.json; analysis/*.csv). The study followed the preregistered stopping rule; no silent adapter substitution occurred.

References

[1] Rafael Cadenas, "Convergent Correctness in Stochastic Code Generation: A Generate-Verify-Repair Architecture for Deterministic Validation of Autonomous AI Coding Agents in Regulated Environments", 2026, 10.2139/ssrn.6754899

[2] https://arxiv.org/abs/2607.20478

[3] Arnav Gupta, "Verifier Exploitation in NLI-Guided Iterative Refinement: A Controlled Empirical Analysis", 2026, 10.21203/rs.3.rs-10187492/v1

[4] http://arxiv.org/abs/2603.19328

[5] Muhammad Bilal, Jon Crowcroft, Ruizhi Wang, Xiaolong Xu, Schahram Dustdar, "Large Language Models for Agentic NetOps and AIOps: Architectures, Evaluation, and Safety", 2026, 10.48550/arxiv.2605.12729